

Evaluacion

May 25, 2026

```
[2]: # 1. Importar librerías
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

# 2. Cargar el dataset
df = pd.read_csv('trafico_red.csv')

# 3. Eliminar columnas no numéricas (IPs, timestamp, etc.)
columnas_a_eliminar = ['src_ip', 'dst_ip', 'timestamp']
for col in columnas_a_eliminar:
    if col in df.columns:
        df = df.drop(col, axis=1)

print("Dataset cargado. Primeras filas:")
print(df.head())
print("\nDistribución de etiquetas:")
print(df['label'].value_counts())

# 4. Preparar los datos
X = df.drop('label', axis=1)
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"\nEntrenamiento: {X_train.shape[0]} muestras")
print(f"Prueba: {X_test.shape[0]} muestras")

# 5. Entrenar y evaluar modelos
modelos = {
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}
```

```

for nombre, modelo in modelos.items():
    modelo.fit(X_train, y_train)
    y_pred = modelo.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    print(f"\n{'='*40}")
    print(f"--- {nombre} ---")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1-Score: {f1:.4f}")
    print("\nReporte de Clasificación:")
    print(classification_report(y_test, y_pred))

```

Dataset cargado. Primeras filas:

	src_port	dst_port	protocol	flow_duration	flow_byts_s	flow_pkts_s	\
0	51913	443	17	25.285889	35.869809	0.395477	
1	5353	5353	17	19.989860	15.457837	0.150076	
2	29097	53	17	0.020400	23333.333333	147.058824	
3	40920	443	6	5.554403	2636.286924	7.921643	
4	49336	443	6	10.577655	177.449539	0.661772	

	fwd_pkts_s	bwd_pkts_s	tot_fwd_pkts	tot_bwd_pkts	...	fwd_blk_rate_avg	\
0	0.237286	0.158191	6	4	...	0.0	
1	0.150076	0.000000	3	0	...	0.0	
2	98.039216	49.019608	2	1	...	0.0	
3	4.140859	3.780784	23	21	...	0.0	
4	0.661772	0.000000	7	0	...	0.0	

	bwd_blk_rate_avg	fwd_seg_size_avg	bwd_seg_size_avg	cwr_flag_count	\
0	0.000000e+00	94.000000	85.750000	0	
1	0.000000e+00	103.000000	0.000000	0	
2	0.000000e+00	80.000000	316.000000	0	
3	1.636425e+07	221.130435	455.095238	0	
4	0.000000e+00	268.142857	0.000000	0	

	subflow_fwd_pkts	subflow_bwd_pkts	subflow_fwd_byts	subflow_bwd_byts	\
0	6	4	564	343	
1	3	0	309	0	
2	2	1	160	316	
3	23	21	5086	9557	
4	7	0	1877	0	

```

    label
0  normal
1  normal
2  normal
3  normal
4  normal

```

[5 rows x 80 columns]

Distribución de etiquetas:

```

label
normal      1681
psiphon      164
Name: count, dtype: int64

```

Entrenamiento: 1291 muestras

Prueba: 554 muestras

=====

--- Random Forest ---

Accuracy: 0.9440

Precision: 0.9398

Recall: 0.9440

F1-Score: 0.9394

Reporte de Clasificación:

	precision	recall	f1-score	support
normal	0.95	0.99	0.97	499
psiphon	0.82	0.56	0.67	55
accuracy			0.94	554
macro avg	0.88	0.77	0.82	554
weighted avg	0.94	0.94	0.94	554

=====

--- Decision Tree ---

Accuracy: 0.9224

Precision: 0.9256

Recall: 0.9224

F1-Score: 0.9239

Reporte de Clasificación:

	precision	recall	f1-score	support
normal	0.96	0.95	0.96	499
psiphon	0.60	0.65	0.63	55

accuracy			0.92	554
macro avg	0.78	0.80	0.79	554
weighted avg	0.93	0.92	0.92	554

=====

--- KNN ---

Accuracy: 0.9170

Precision: 0.9040

Recall: 0.9170

F1-Score: 0.9052

Reporte de Clasificación:

	precision	recall	f1-score	support
normal	0.93	0.98	0.96	499
psiphon	0.66	0.35	0.45	55
accuracy			0.92	554
macro avg	0.79	0.66	0.70	554
weighted avg	0.90	0.92	0.91	554

[]: